

Volume 4 – Database Design Document (DDD)

AI Quant Trading Platform

Version: 1.0

Document Type: Database Design Document

1. Overview

Purpose

This document defines the logical and physical database architecture for the AI Quant Trading Platform.

The database must support:

- Multi-tenant SaaS architecture
- High-frequency trading metadata
- AI scoring
- Market analysis
- Portfolio management
- Reporting
- Audit logging
- Strategy management
- Background workers

Primary Database

- PostgreSQL

Caching

- Redis

Future Analytics

- ClickHouse (optional)

Future Time-Series

- TimescaleDB (optional)
-

2. Database Design Principles

- Third Normal Form (3NF) for transactional data
 - Denormalized analytical views where appropriate
 - Soft delete support
 - Audit trail for critical entities
 - UUID primary keys
 - Created/Updated timestamps
 - Optimistic locking where required
 - Row-level tenant isolation
-

3. Multi-Tenant Design

Every business table contains:

- tenant_id
- created_at
- updated_at
- created_by
- updated_by
- deleted_at (nullable)

This ensures complete separation of customer data.

4. Core Database Domains

Identity & Access

- users
 - user_profiles
 - roles
 - permissions
 - role_permissions
 - user_roles
 - sessions
 - refresh_tokens
 - devices
 - api_keys
 - audit_logs
-

Tenant Management

- tenants
- tenant_settings
- subscription_plans

- subscriptions
 - invoices
 - payments
 - coupons
 - usage_metrics
-

Exchange Integration

- exchanges
 - exchange_accounts
 - exchange_api_credentials
 - exchange_permissions
 - exchange_connections
 - exchange_logs
 - websocket_sessions
-

Market Data

- markets
 - trading_pairs
 - assets
 - tickers
 - ohlcv_1m
 - ohlcv_5m
 - ohlcv_15m
 - ohlcv_1h
 - ohlcv_4h
 - ohlcv_1d
 - order_books
 - recent_trades
 - funding_rates
 - open_interest
 - liquidations
 - dominance_metrics
-

Scanner Engine

- scanner_jobs
 - scanner_results
 - candidate_coins
 - rejected_coins
 - scanner_filters
 - scanner_history
-

Technical Analysis

- indicator_sets
 - ema_values
 - sma_values
 - rsi_values
 - macd_values
 - atr_values
 - bollinger_values
 - volume_profile
 - support_levels
 - resistance_levels
 - trend_analysis
 - market_structure
-

AI Engine

- ai_models
 - ai_versions
 - ai_predictions
 - ai_scores
 - ai_reasoning
 - ai_confidence
 - ai_features
 - ai_training_jobs
 - ai_learning_dataset
 - ai_feedback
-

Strategy Engine

- strategies
 - strategy_versions
 - strategy_parameters
 - strategy_conditions
 - strategy_assignments
 - strategy_results
 - custom_strategies
-

Risk Engine

- risk_profiles
- risk_rules
- exposure_limits
- drawdown_history
- position_limits
- daily_loss_limits
- emergency_stop_events

Trading Engine

- trade_signals
- signal_history
- signal_rejections
- orders
- order_events
- order_status_history
- executions
- positions
- position_history
- stop_loss_rules
- take_profit_rules
- trailing_stop_events

Portfolio

- portfolios
- portfolio_balances
- asset_allocations
- portfolio_history
- pnl_history
- realized_pnl
- unrealized_pnl

Notifications

- notification_channels
- notification_templates
- notifications
- notification_logs
- delivery_status

Reporting

- reports
- report_exports
- trade_statistics
- performance_metrics
- monthly_summaries
- yearly_summaries

Paper Trading

- paper_accounts

- paper_orders
 - paper_positions
 - paper_trades
 - paper_reports
-

Backtesting

- backtest_jobs
 - backtest_results
 - optimization_runs
 - strategy_rankings
-

Administration

- admin_users
 - system_settings
 - feature_flags
 - maintenance_logs
 - system_events
-

Monitoring

- worker_status
 - queue_metrics
 - api_metrics
 - service_health
 - performance_logs
-

5. Common Table Structure

Every transactional table should include:

- id (UUID)
 - tenant_id
 - created_at
 - updated_at
 - created_by
 - updated_by
 - version
 - deleted_at (nullable)
-

6. Key Relationships

Tenant — Users

Users — Exchange Accounts — Strategies — Risk Profiles — Portfolios — Notifications

Exchange Account — Orders — Positions — Balances

Strategy — Trade Signals — AI Scores — Backtests

Order — Executions — Position — Order Events

Position — Stop Loss — Take Profit — PnL History

7. Indexing Strategy

Primary Indexes

- id
- tenant_id

Secondary Indexes

- user_id
- exchange_id
- strategy_id
- trading_pair
- status
- created_at
- updated_at

Composite Indexes

- tenant_id + created_at
 - tenant_id + user_id
 - exchange_id + symbol
 - strategy_id + status
-

8. Partitioning Strategy

Large tables should be partitioned by date:

- ohlcv
- recent_trades

- order_events
- executions
- notifications
- audit_logs
- scanner_results

Monthly partitions recommended.

9. Caching Strategy

Redis caches:

- Market prices
 - Indicator values
 - User sessions
 - AI scores
 - Exchange balances
 - Frequently accessed settings
-

10. Audit Logging

Every sensitive action must create an audit record.

Track:

- Login
 - Logout
 - Password changes
 - API key updates
 - Exchange connections
 - Strategy changes
 - Risk profile updates
 - Order submissions
 - Order cancellations
 - Admin actions
-

11. Data Retention

Market Data

- High-frequency ticks: configurable retention
- OHLCV: long-term

Trade History

- Permanent

Audit Logs

- Minimum retention as configured by administrator

Reports

- Permanent unless deleted by user
-

12. Backup Strategy

- Daily incremental backups
 - Weekly full backups
 - Encrypted backup storage
 - Point-in-time recovery
 - Backup verification jobs
-

13. Migration Strategy

- Version-controlled migrations
 - Zero-downtime deployment support
 - Rollback scripts for each release
 - Data validation after migration
-

14. Performance Guidelines

- Use pagination for large result sets
 - Batch writes where possible
 - Async processing for heavy operations
 - Avoid N+1 queries
 - Prefer indexed lookups
 - Archive historical analytical data
-

15. Future Database Enhancements

- Time-series optimization with TimescaleDB
- Analytical warehouse using ClickHouse
- Read replicas for reporting
- Cross-region replication
- Event sourcing for trade lifecycle

- Data lake for machine learning datasets
 - Vector database for AI embeddings (optional)
-

16. Naming Conventions

- Table names: plural, snake_case
 - Column names: snake_case
 - Foreign keys: <entity>_id
 - Boolean fields: is_ or has_
 - Timestamps: *_at
 - Status fields: ENUM or constrained lookup tables
-

17. Security Rules

- Encrypt API credentials at rest
- Never store exchange secrets in plaintext
- Apply row-level authorization through tenant_id
- Record every privileged action
- Restrict direct database access to application services
- Validate all inputs before persistence